# A Blockchain-Based Ticketing Architecture for Preventing Ticket Fraud and Scalping in Kenya

**[INTERNAL PROJECT]**

**Leriari Peter Tobiko**

**166913**

**ICS 4B**

**Supervisor Name**

**Dr. Vincent Ochango**

**An Informatics and Computer Science Project Documentation Submitted to the School of Computing and Engineering Sciences in Partial Fulfillment of the Requirements for the Award of a Degree in Bachelor of Science in Informatics and Computer Science**

**School of Computing and Engineering Sciences**

**Strathmore University**

**Nairobi, Kenya**

**June 2026**

**Declaration and Approval**

I declare that this work has not been previously submitted and approved for the award of a degree by this or any other University. To the best of my knowledge and belief, the research documentation contains no material previously published or written by another person except where due reference is made in the research proposal itself.

Student Name: Leriari Peter Tobiko

Admission Number: 166913

Student Signature:  Date: 11th June 2026

The Report of **Leriari Peter Tobiko** has been reviewed and approved by **Dr. Vincent Ochango**

Supervisor Signature:  Date: 11th June 2026

**Abstract**

Event ticketing systems continue to face challenges related to ticket fraud, ticket scalping, and unreliable ticket validation, particularly in emerging markets where mobile-money payments dominate and internet connectivity cannot always be guaranteed. Although blockchain-based ticketing solutions have demonstrated the potential to improve ticket ownership transparency and enforce resale controls through smart contracts, existing architectures largely target cryptocurrency-native environments and provide limited support for mobile-money integration and offline validation. Consequently, a gap exists between the capabilities of current blockchain ticketing solutions and the operational requirements of markets such as Kenya. This study investigated how a blockchain-based ticketing architecture can be designed to address these limitations while preserving the security and transparency benefits of decentralized ownership.

To address the identified gap, the study evaluated existing blockchain-based ticketing solutions and proposes ChainTix, a ticketing architecture that integrates mobile-money payments, blockchain-backed ticket ownership, smart contract-enforced resale controls, and offline ticket validation mechanisms. The architecture was implemented and evaluated against the limitations identified in existing solutions, with particular focus on ticket authenticity, resale governance, validation reliability, and accessibility for non-cryptocurrency users. The study contributes to the growing body of research on blockchain-based ticketing by demonstrating how decentralized ticket ownership models can be adapted to emerging-market environments characterized by widespread mobile-money adoption and intermittent network connectivity.

***Keywords: Anti-Scalping, Blockchain Ticketing, Dynamic QR Codes, Event Ticketing, M-Pesa, NFT, Smart Contracts, TOTP***

## Table of Contents

Declaration and Approval ..... ii

Abstract ..... iii

List of Figures ..... vii

List of Tables ..... viii

List of Abbreviations ..... ix

Chapter 1: Introduction ..... 1

1.1 Background Information ..... 1

1.2 Problem Statement ..... 1

1.3 Objectives ..... 2

1.3.1 General Objective ..... 2

1.3.2 Specific Objectives ..... 2

1.4 Research Questions ..... 2

1.5 Justification ..... 3

1.6 Scope ..... 3

1.7 Limitations and Delimitations ..... 4

1.7.1 Limitations ..... 4

1.7.2 Delimitations ..... 4

Chapter 2: Literature Review ..... 6

2.1 Introduction ..... 6

2.2 Current State of Event Ticketing in Kenya ..... 6

2.2.1 Technical Challenges Facing Event Ticketing in Kenya ..... 7

2.3 Related Works ..... 8

2.3.1 SafeTicket: Smart Contract-Enforced Secondary Markets ..... 8

2.3.2 TickEth: Open-Specification Cryptographic Asset Management ..... 8

2.3.3 Sui Blockchain and zkLogin: Identity Abstraction ..... 9

2.3.4 OPEN Ticketing Ecosystem (Formerly GET Protocol) ..... 9

2.4 Gaps in Existing Ticketing Solution ..... 9
2.5 Conceptual Framework ..... 11
Chapter 3: Development Methodology ..... 13
3.1 Introduction ..... 13
3.2 System Development Methodology ..... 13
3.2.1 Justification of the Methodology ..... 13
3.3 Methodology Diagram ..... 14
3.3.1 Product Goal ..... 14
3.3.2 Product Backlog ..... 14
3.3.3 Sprint Planning ..... 14
3.3.4 Sprint Backlog and Development Sprints ..... 15
3.3.5 Daily Scrum ..... 15
3.3.6 Sprint Review ..... 15
3.3.7 Sprint Retrospective ..... 15
3.4 Analysis Diagrams ..... 15
3.4.1 Use Case Diagram ..... 15
3.4.2 Class Diagram ..... 16
3.4.3 Sequence Diagram ..... 16
3.4.4 Activity Diagram ..... 16
3.4.5 Package Diagram ..... 17
3.5 Design Diagrams ..... 17
3.5.1 System Architecture Diagram ..... 17
3.5.2 Database Design ..... 18
3.5.3 Wireframes ..... 18
3.6 Deliverables ..... 18
3.6.1 System Documentation ..... 18
3.6.2 Mobile Application ..... **Error! Bookmark not defined.**

3.6.3 Smart Contracts ..... 19
3.6.4 Administrative Dashboard ..... 19
3.7 Tools and Techniques ..... 19
3.7.1 React Native ..... **Error! Bookmark not defined.**
3.7.2 React Web ..... 19
3.7.3 FastAPI ..... 20
3.7.4 PostgreSQL ..... 20
3.7.5 Base Blockchain ..... 20
3.7.6 Solidity ..... 20
3.7.7 M-Pesa Daraja API ..... 20
3.7.8 Time-Based One-Time Password (TOTP) ..... 20
References ..... 36
Appendix: ..... 38
Appendix 1: Gantt Chart ..... 38

## List of Figures

Figure 2.1 Conceptual Framework ..... 12
Figure 3.1 Scrum Framework (Scrum.org, 2026) ..... 14

## List of Tables

Table 2.1 Comparative Analysis of Reviewed Blockchain Ticketing Systems.....11

## List of Abbreviations

DLT - Distributed Ledger Technology

ERC - Ethereum Request for Comment

ETH - Ether

HMAC - Hash-Based Message Authentication Code

IPFS - InterPlanetary File System

NFT - Non-Fungible Token

TOTP- Time-based One Time Password

zkLogin - Zero-Knowledge Login

# Chapter 1: Introduction

## 1.1 Background Information

Event ticketing has evolved from paper-based systems to digital platforms capable of processing transactions at scale. While barcode and QR-code ticketing improved convenience and accessibility, they remain vulnerable to duplication, screenshot fraud, and unregulated resale. Recent advances in blockchain technology and Non-Fungible Tokens (NFTs) have introduced new possibilities for establishing cryptographically verifiable ticket ownership, enabling transparent ticket issuance, transfer, and validation through immutable digital records (Gómez et al., 2024). The growing economic significance of the global ticketing industry, valued at approximately USD 78.2 billion in 2024, has further increased the need for secure and trustworthy ticketing infrastructures (SNS Insider, 2025).

In Kenya, the adoption of blockchain-based ticketing is constrained by low cryptocurrency usage despite widespread mobile-money adoption. M-Pesa serves over 34 million active subscribers and remains the dominant digital payment platform (Safaricom PLC, 2024) making many cryptocurrency-dependent ticketing solutions unsuitable for the local market. Existing studies and commercial deployments have demonstrated the feasibility of blockchain-backed ticket ownership and smart contract-based resale governance (Avalanche, 2025; Dhanushkodi, 2025; Gómez et al., 2024), yet provide limited consideration for mobile-money integration, offline ticket validation, and emerging-market constraints. This study therefore, investigates how a Web 2.5 blockchain-based ticketing architecture can address these challenges while preserving the security and transparency benefits of decentralized ownership.

## 1.2 Problem Statement

Event ticketing systems continue to face challenges related to ticket fraud, ticket scalping, and unreliable ticket validation, resulting in financial losses for event organisers and reduced consumer trust. In Kenya, the Burna Boy concert in Nairobi highlighted the consequences of counterfeit tickets and secondary market abuse, where legitimate attendees were denied entry despite available seating (Tiko Hub, 2025). Globally, incidents of ticket fraud continue to rise, with reports indicating substantial financial losses associated with counterfeit ticket sales and unauthorised resale activities (Audience View, 2026; Soft Journ, 2026).

Although blockchain-based ticketing systems have demonstrated the potential to improve ticket ownership transparency and enforce resale controls through smart contracts (Dhanushkodi, 2025; Gómez et al., 2024), existing architectures largely target cryptocurrency-

native environments and assume continuous internet connectivity. These assumptions limit their suitability in emerging markets such as Kenya, where mobile-money payments dominate digital transactions (Safaricom PLC, 2024) and network reliability at event venues cannot always be guaranteed. Furthermore, existing solutions typically address isolated aspects of the ticketing problem, such as ownership transparency or resale control, but provide limited support for the combined requirements of mobile-money integration, offline validation, and secondary market governance. Consequently, there is a need to investigate how blockchain-based ticketing architectures can be adapted to support mobile-money integration, offline ticket validation, and secure ticket ownership while addressing the challenges of fraud and scalping within the Kenyan event ticketing ecosystem.

## 1.3 Objectives

### 1.3.1 General Objective

To design and evaluate the effectiveness of a blockchain-based ticketing architecture in addressing ticket fraud, scalping, and validation challenges in Kenya.

### 1.3.2 Specific Objectives

i. To examine the current state of event ticketing in Kenya and the challenges associated with fraud, scalping, and ticket validation.
ii. To evaluate the architectural approaches employed by existing blockchain-based and NFT-backed ticketing systems.
iii. To identify the architectural limitations of existing architectures in supporting mobile-money integration, secondary market governance, and offline ticket validation.
iv. To design and implement a blockchain-based ticketing architecture that addresses the identified limitations.
v. To evaluate the effectiveness of the Web 2.5 ticketing architecture in addressing the limitations of existing ticketing solutions.

## 1.4 Research Questions

i. What is the current state of event ticketing in Kenya, and what challenges are associated with ticket fraud, scalping, and ticket validation?
ii. How have existing blockchain-based ticketing systems been developed and applied, and what are their strengths and limitations?
iii. What are the key gaps and challenges in decentralized ticketing research regarding mobile money integration, anti-scalping enforcement, and offline ticket validation?

iv. How can a blockchain-based ticketing architecture be designed to support secure ticket purchasing, resale, and validation in the Kenyan market?
v. How does the proposed blockchain-based ticketing architecture address the challenges of ticket fraud, ticket scalping, and ticket validation identified in existing ticketing systems?

## 1.5 Justification

Existing research has demonstrated the potential of blockchain technology to improve ticket ownership transparency, automate resale governance, and reduce ticket fraud through smart contracts and NFT-based ownership models. Studies such as SafeTicket (Gómez et al., 2024) and blockchain-based ticketing frameworks proposed by Dhanushkodi (2025) have established the technical feasibility of decentralized ticket management, while commercial deployments have demonstrated its practical viability at scale. However, the majority of existing solutions have been designed for cryptocurrency-native environments and provide limited consideration for emerging markets where mobile-money payments dominate and internet connectivity cannot always be guaranteed.

A review of the literature reveals persistent gaps relating to mobile-money integration, offline ticket validation, and the applicability of blockchain ticketing architectures within low-cryptocurrency-adoption environments. These limitations reduce the suitability of existing solutions for markets such as Kenya. Consequently, there is a need to investigate how blockchain-based ticketing architectures can be adapted to support local payment ecosystems while maintaining the security, transparency, and governance benefits of decentralized ownership. This study is justified on the basis that it contributes knowledge on the design and evaluation of a Web 2.5 ticketing architecture tailored to emerging-market contexts, thereby extending current research beyond cryptocurrency-centric implementations.

## 1.6 Scope

This study focused on the design and evaluation of a Web 2.5 blockchain-based ticketing architecture for the Kenyan event ticketing ecosystem. The study investigated the integration of mobile-money payments, blockchain-backed ticket ownership, smart contract-enforced resale controls, and offline ticket validation mechanisms within a unified framework. The evaluation was conducted with reference to the challenges identified in existing ticketing systems, namely ticket fraud, ticket scalping, and ticket validation reliability.

The study was limited to the Kenyan context and considered M-Pesa as the primary payment ecosystem. The architecture is evaluated based on its ability to address the identified gaps in existing solutions and did not seek to assess broader regulatory, legal, or cryptocurrency adoption issues beyond those directly related to ticketing. The project encompassed requirements analysis, architecture design, implementation, testing, evaluation, and documentation within the duration allocated for the study.

## 1.7 Limitations and Delimitations

### 1.7.1 Limitations

The study evaluated the architecture within a controlled development environment rather than a live event setting. Consequently, findings relating to ticket validation performance, user interaction, and operational scalability may not fully reflect conditions encountered during large-scale events.

The evaluation relied on the Safaricom Daraja sandbox environment, which does not replicate all behaviours of the production M-Pesa ecosystem. As a result, certain real-world payment processing scenarios, including network congestion and service disruptions, may not be fully represented.

The offline validation mechanism was assessed through simulated testing. Although the proposed approach incorporates tolerance windows to accommodate clock variations, the study did not evaluate all possible device synchronization scenarios that may arise in operational deployments.

Additionally, the study focused on the architectural suitability of the proposed solution and did not include a large-scale user study. Therefore, conclusions relating to user acceptance and adoption were limited to observations derived from system testing and architectural evaluation.

### 1.7.2 Delimitations

This study was limited to the design and evaluation of a Web 2.5 blockchain-based ticketing architecture within the Kenyan event ticketing context. The investigation focused specifically on mobile-money integration, blockchain-backed ticket ownership, smart contract-based resale governance, and offline ticket validation mechanisms.

The study considered M-Pesa as the primary payment ecosystem and did not evaluate integration with other mobile-money providers. Similarly, the architecture is implemented on

the Base blockchain and did not include a comparative evaluation of alternative blockchain networks.

The scope of the study was restricted to the technical and architectural aspects of event ticketing. Issues relating to business models, regulatory frameworks, market adoption strategies, and broader cryptocurrency policy considerations fell outside the scope of this research.

Furthermore, the evaluation focused on the effectiveness of the proposed architecture in addressing the identified gaps in existing ticketing solutions and did not include a commercial deployment or operational pilot event.

# Chapter 2: Literature Review

## 2.1 Introduction

This chapter presents a critical review of literature related to event ticketing systems, blockchain-based ticket ownership, smart contract governance, and ticket validation mechanisms. The review examines the current state of event ticketing in Kenya, evaluates existing blockchain-based and NFT-backed ticketing architectures, and identifies the limitations that hinder their applicability in emerging-market environments. The chapter further synthesizes the identified gaps and presents the conceptual framework that guided the design and evaluation of the proposed blockchain-based ticketing architecture.

## 2.2 Current State of Event Ticketing in Kenya

The global online event ticketing market was valued at approximately USD 78.2 billion in 2024 and is projected to expand significantly through 2032, underscoring the economic scale of the ecosystem this study seeks to secure (SNS Insider, 2025). In Kenya, the industry has grown rapidly alongside rising urbanisation and entertainment spending; however, the platforms serving this market remain architecturally vulnerable to three compounding failure modes: ticket counterfeiting, unregulated secondary market scalping, and gate-level validation failures that collectively undermine the financial viability of event operations.

Event ticketing has evolved through four broadly identifiable phases. Physical paper tickets dominated the first phase, offering simplicity but no protection against counterfeiting beyond the physical difficulty of replication. The second phase introduced barcode-based digital tickets distributed via email, enabling online sales at scale but introducing screenshot duplication vulnerabilities. The third phase saw the adoption of QR codes, which improved scannability but remained equally susceptible to screenshot capture and redistribution. The fourth and current phase is characterised by mobile ticketing applications, digital wallets, and, in a growing number of markets, blockchain-based NFT tickets, introducing for the first time the possibility of cryptographically unique, tamper-proof ticket assets that cannot be duplicated regardless of how they are shared.

In Kenya, this evolution has proceeded unevenly against a market structure shaped by high mobile money penetration and low cryptocurrency literacy. M-Pesa serves over 34 million active subscribers and commands a market penetration exceeding 91% among mobile payment users (Safaricom PLC, 2024), meaning any ticketing solution requiring credit card payments or cryptocurrency ownership is structurally inaccessible to the majority of Kenyan event-goers.

QR-based digital tickets distributed via email and SMS dominate the mainstream market, with platforms such as Mookh, TicketSasa, and Quicket providing online payment processing but no blockchain backing, no smart contract enforcement, and no offline validation capability. A review of available literature and commercial deployments found limited evidence of blockchain-based ticketing adoption within the Kenyan market, representing a gap that motivates further investigation.

### 2.2.1 Technical Challenges Facing Event Ticketing in Kenya

Counterfeiting and static ticket vulnerabilities are the dominant digital ticketing models in Kenya distributes static QR codes and PDF tickets via email or SMS. Because these assets can be screenshot and shared across messaging platforms without any cryptographic mechanism to detect duplication, they are structurally incapable of guaranteeing uniqueness at the gate. The consequences of this vulnerability were empirically demonstrated at the Burna Boy concert at Uhuru Gardens in Nairobi on March 1, 2025. Despite the event being officially sold out, the venue exhibited a significant number of unoccupied seats while thousands of fans were denied entry. This outcome was directly attributed to two interrelated factors: bulk-hoarding by scalpers who failed to liquidate their inventory, and the widespread circulation of counterfeit ticket screenshots on social media platforms, particularly TikTok. Legitimate purchasers who acquired tickets through unverified secondary channels suffered financial losses, while the organizer faced reputational damage and crowd control liabilities (Tiko Hub, 2025).

Venue network congestion and validation failure. Traditional electronic ticketing systems require continuous internet connectivity at the point of gate validation, executing real-time queries against a centralized relational database to verify ticket state. At large-scale Kenyan venues such as Kasarani Stadium or Nyayo National Stadium, the convergence of tens of thousands of attendees creates severe cellular network congestion. Under these conditions, scanning devices experience high latency, packet loss, and complete communication timeouts. Gate operators are then forced into an unsustainable choice: suspended entry, causing dangerous crowd accumulation, or admit attendees based on visual inspection of static tickets, which inevitably admits duplicates. Platforms such as GiG have introduced real-time fraud detection at the application layer, but because these systems lack cryptographically signed ownership or offline mathematical validation capability, they remain dependent on active network access and fail under the same congestion conditions (Miguel Faustino Oliveira et al., 2021).

Unregulated secondary markets and algorithmic scalping. The absence of protocol-level controls on secondary market transactions enables speculative intermediaries to exploit ticket scarcity on a large scale. The pattern extends beyond music events: the official digital ticketing portal for the African Nations Championship (CHAN), managed by Mookh Africa under CAF oversight, was systematically exploited because the centralized system lacked purchase thresholds or identity-binding constraints per user account. Politically connected networks bulk-purchased VIP and terrace allocations, leaving genuine fans unable to enter a 48,000-capacity venue that was visibly under-attended.

## 2.3 Related Works

The following subsections critically evaluate four blockchain-based and NFT-backed ticketing solutions. Each is assessed for its technology stack, strengths, and limitations relative to the Kenyan market requirements identified in Section 2.2. Table 2.1 provides a structured comparative summary.

### 2.3.1 SafeTicket: Smart Contract-Enforced Secondary Markets

The SafeTicket framework, proposed by (Gómez et al., 2024), issues ERC-721 NFT tickets on the Ethereum blockchain and encodes resale price caps and creator royalty distributions directly into the smart contract's execution logic. An empirical evaluation with 70 participants drawn from a concert-going population of 30,000 in Lima, Peru, found that 60% of respondents perceived the framework to offer high security guarantees. SafeTicket is methodologically significant because it constitutes the first peer-reviewed demonstration of protocol-level secondary market enforcement through smart contracts. Its critical limitation is the assumption that users possess cryptocurrency wallets and Ether balances to cover gas fees, which renders the platform inaccessible in emerging markets with low cryptocurrency literacy.

### 2.3.2 TickEth: Open-Specification Cryptographic Asset Management

TickEth, proposed by Corsi et al. (2019) defines an open, client-agnostic specification for blockchain ticketing that addresses Sybil attacks by binding each Ethereum wallet address to a verified real-world identity through a centralised registration phase. This prevents scalpers from registering multiple accounts to bypass purchase thresholds. While TickEth's identity-binding mechanism is architecturally robust, the centralised registration server introduces a single point of failure, inconsistent with the decentralisation objective of blockchain systems. Furthermore, all financial settlements remain denominated in Ether, limiting accessibility in fiat-dominant markets.

### 2.3.3 Sui Blockchain and zkLogin: Identity Abstraction

SUTHIR et al. (2024) proposes a high-throughput event management platform on the Sui blockchain, leveraging its object-centric data model and parallel transaction consensus to mint and transfer thousands of NFT tickets simultaneously. The platform integrates Sui's zkLogin protocol, enabling non-technical users to access decentralised assets using standard OpenID Connect credentials such as Google or Apple accounts, generating zero-knowledge proofs to secure key custody without requiring seed phrase management. This approach represents a significant advance in user onboarding. However, the validation of zkLogin-derived wallet signatures still requires active network access to verify zero-knowledge proofs, limiting utility at congested physical venues.

### 2.3.4 OPEN Ticketing Ecosystem (Formerly GET Protocol)

The OPEN Ticketing Ecosystem, formerly known as GET Protocol, is one of the most mature commercial implementations of blockchain-based ticketing infrastructure. Established in 2016 and rebranded in 2024, the platform provides on-chain ticket issuance, NFT-backed ownership records, and blockchain-based ticket management tools designed to improve ticket authenticity, ownership transparency, and secondary market control. A key strength of OPEN is its ability to maintain a verifiable ownership history for every ticket through blockchain records, while also supporting ticket claiming, which allows tickets to be converted into collectable NFT assets after an event. The platform has demonstrated the practical viability of blockchain ticketing by issuing millions of tickets for concerts, festivals, and sporting events. However, from the perspective of the Kenyan market, several limitations remain. The ecosystem relies on blockchain-native infrastructure and token-based economics, introducing complexity for users unfamiliar with cryptocurrency systems. Furthermore, there is limited evidence of integration with mobile-money payment platforms such as M-Pesa, and ticket verification remains dependent on blockchain-connected infrastructure, with no documented support for fully offline cryptographic validation. These limitations reduce its suitability for emerging markets where mobile-money payments dominate and reliable internet connectivity cannot always be guaranteed (OPEN Ticketing Ecosystem, 2025).

## 2.4 Gaps in Existing Ticketing Solution

A critical synthesis of the systems reviewed in Section 2.3 and summarised in Table 2.1 reveals four persistent structural gaps that collectively prevent any existing solution from being viable in the Kenyan market.

The most fundamental gap across all reviewed works is the absence of mobile money integration. SafeTicket requires users to hold Ether balances sufficient to cover both ticket costs and gas fees (Gómez et al., 2024), TickEth denominates all financial settlements in Ether through a centralised registration gateway (Corsi et al., 2019), and the Sui-based platform, despite abstracting wallet access through zkLogin, still assumes users transact in the network's native token (SUTHIR et al., 2024). In Kenya, where M-Pesa serves over 34 million active subscribers and formal banking penetration remains low, this uniform assumption of cryptocurrency liquidity represents the single most significant barrier to adoption for any of these systems.

No reviewed system supports gate validation without an active internet connection. As summarised in Table 2.1, SafeTicket and TickEth require real-time state queries against the Ethereum network at the point of entry, while the Sui-based platform requires active network access to verify its zkLogin zero-knowledge proofs. As documented in Section 2.2.2, venue-level network congestion at large Kenyan venues renders all three systems operationally unviable under real-world conditions, yet none proposes a mechanism for offline mathematical validation.

Protocol-level anti-scalping enforcement remains incomplete across the reviewed works. SafeTicket is the only system to demonstrate smart contract-enforced resale price caps, and TickEth partially addresses bulk purchasing through identity-bound account thresholds, while the Sui-based platform introduces no resale enforcement mechanism at all. Critically, both SafeTicket and TickEth assume that secondary transactions are settled in cryptocurrency on a public ledger, making on-chain price caps enforceable. In the Kenyan context, scalpers can trivially circumvent these controls by settling payments off-chain via cash or peer-to-peer M-Pesa transfers while transferring the NFT on-chain for a nominal fee. No reviewed work addresses this misalignment between on-chain ownership transfer and off-chain payment settlement.

Finally, all three systems treat the user's mobile device as a passive display terminal, neglecting client-side security entirely. SafeTicket, TickEth, and the Sui-based platform each assume that rendering a ticket credential on screen is sufficient for gate presentation, without addressing the straightforward attack vector of screenshot capture and redistribution. If a static ticket view can be screenshotted, the cryptographic guarantees of the underlying NFT are bypassed at

precisely the point that matters most, yet none of the reviewed architectures implement screen capture prevention or device integrity detection.

Table 2.1 Comparative Analysis of Reviewed Blockchain Ticketing Systems

| System | Blockchain | Payment Method | Anti-Scalping | Offline Validation | Mobile Money Support | Key Limitation |
|---|---|---|---|---|---|---|
| SafeTicket(Gomez et al., 2024) | Ethereum (L1) | Cryptocurrency | Smart contract cap | No | No | Requires crypto wallet and ETH gas fees |
| TickEth(Corsi et al., 2019) | Ethereum (L1) | Cryptocurrency | Identity-bound limits | No | No | Centralised identity registration point |
| Sui/zkLogin(Sriram et al., 2025) | Sui (L1) | Cryptocurrency (SUI) | Smart contract rules | No | No | ZK proof verification needs network access |
| OPEN Ticketing Ecosystem | Multi-chain (Polygon/Base) | Cryptocurrency / Tokenized ecosystem | Secondary market controls | No | No | No mobile-money integration |
| ChainTix (This Study) | Base (L2) | M-Pesa (KES) | Smart contract cap | Yes (TOTP) | Yes | Custodial wallet trust model |

## 2.5 Conceptual Framework

Figure 2.1 illustrates the conceptual framework underpinning the ChainTix platform. The framework was derived from the gaps identified in the literature review, namely ticket fraud,

ticket scalping, limited support for mobile-money payment systems, and dependence on continuous internet connectivity during ticket validation.

The framework begins with ticketing activities initiated by event organizers and attendees. Organizers create events and define ticketing rules, while attendees submit ticket purchase requests using mobile-money payment services. These inputs are processed through a secure ticketing mechanism consisting of payment verification, blockchain-based ticket issuance, smart contract enforcement, and ownership management. Upon successful payment verification, a unique NFT-backed ticket is issued and linked to the purchaser. Smart contracts govern ticket ownership and enforce resale policies, including resale restrictions and royalty distribution rules where applicable.

To support event admission, the framework incorporates a ticket validation process that generates time-sensitive validation credentials for each ticket. These credentials enable both online and offline verification during event entry, reducing dependence on continuous internet connectivity. Ticket ownership records, validation activities, and resale transactions are maintained in a transparent and traceable manner throughout the ticket lifecycle.

The interaction of these processes produces authentic and verifiable tickets, controlled ticket resale, transparent ownership records, and reliable ticket validation. Ultimately, the framework aims to reduce ticket fraud and scalping while improving trust, accessibility, and operational efficiency within the event ticketing ecosystem.

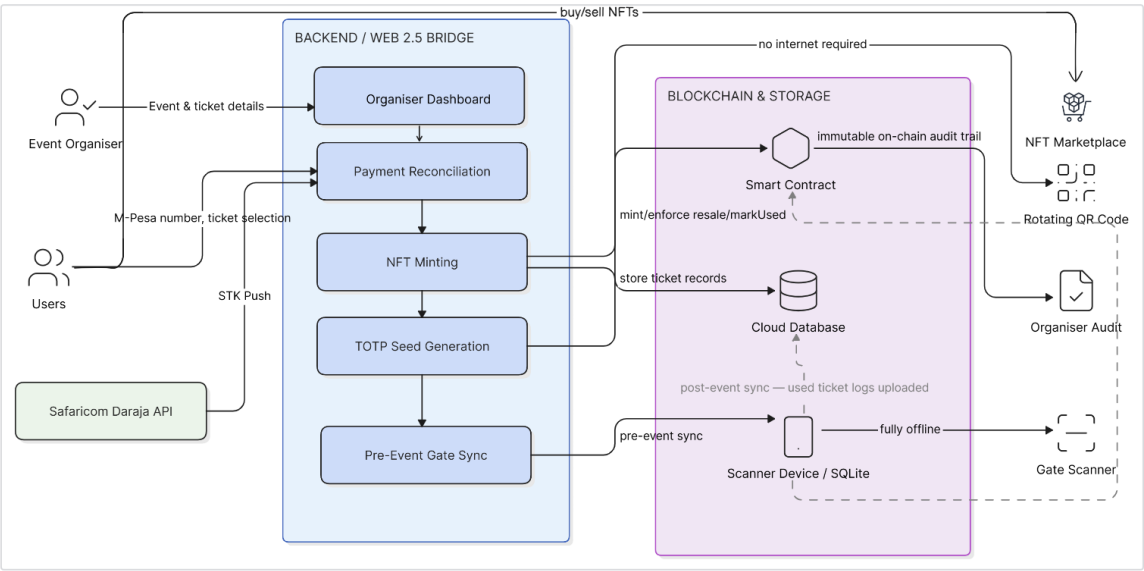



Figure 2.1 Conceptual Framework

# Chapter 3: Development Methodology

## 3.1 Introduction

This chapter presents the methodology adopted in the design and development of the proposed ChainTix platform. It outlines the development paradigm, software development methodology, implementation approach, testing strategy, analysis and design models, and the tools and technologies used throughout the project. The chapter also describes the deliverables produced by the study. The methodology was selected to support the development of a secure blockchain-based ticketing architecture capable of addressing the challenges identified in Chapter Two, namely ticket fraud, scalping, and offline ticket validation within the Kenyan market.

## 3.2 System Development Methodology

This study adopted the Agile software development methodology, specifically the Scrum framework, to guide the development of the proposed ChainTix platform. Agile Scrum was selected because the system comprises several interconnected components, including a progressive web application, backend services, blockchain smart contracts, M-Pesa payment integration, and offline ticket validation. The methodology supports iterative development through short development cycles known as sprints, allowing system features to be implemented, tested, and refined incrementally (Alami & Krancher, 2022).

The project adopted Object-Oriented Analysis and Design (OOAD) as its development paradigm. OOAD was suitable because the system can be modeled as interacting objects such as users, events, tickets, payments, and smart contracts. The combination of Agile Scrum and OOAD supports modular development, continuous testing, and flexibility throughout the project lifecycle.

### 3.2.1 Justification of the Methodology

Agile Scrum was selected because it provides a flexible and iterative approach to developing complex software systems. The ChainTix platform integrates multiple technologies and functionalities that require continuous development, testing, and refinement. Scrum enabled these components to be implemented incrementally while reducing the risks associated with late-stage integration.

The methodology also supported continuous evaluation through sprint reviews and retrospectives, allowing improvements to be introduced throughout development. In addition, OOAD complemented Scrum by providing a structured way of modeling system components and their interactions. Therefore, the combination of Agile Scrum and OOAD was appropriate

for the design, implementation, and evaluation of the proposed blockchain-based ticketing platform.

## 3.3 Methodology Diagram

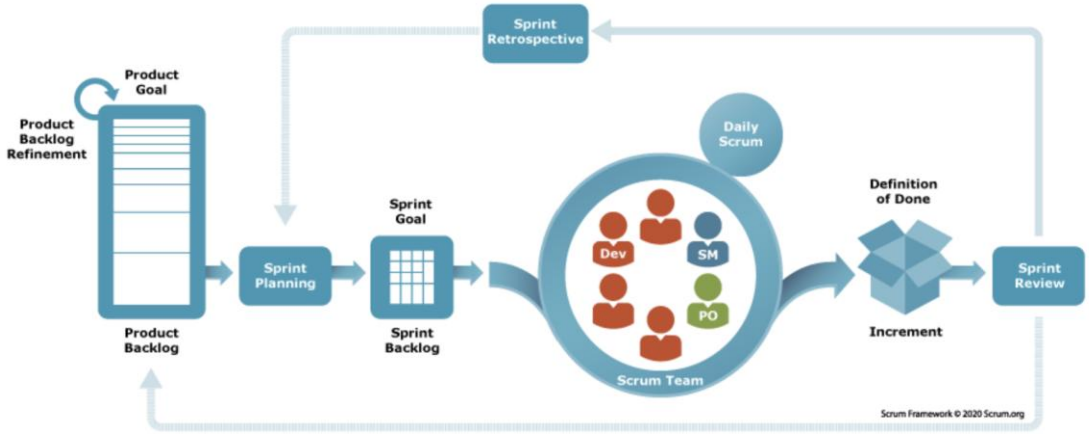



*Figure 3.1 Scrum Framework (Scrum.org, 2026)*

### 3.3.1 Product Goal

The overall product goal of the project was to design and develop a secure blockchain-based ticketing platform that addresses ticket fraud, ticket scalping, and ticket validation challenges within the Kenyan event ticketing ecosystem. The platform integrated M-Pesa payments, NFT-backed ticket ownership, smart contract-based resale controls, and offline ticket validation mechanisms within a unified Web 2.5 architecture.

### 3.3.2 Product Backlog

The Product Backlog represented a prioritized list of functional and non-functional requirements derived from the literature review and system analysis activities. These requirements defined the capabilities that had to be implemented to achieve the objectives of the study. The backlog remained dynamic throughout development and was refined as new requirements or implementation considerations emerge.

### 3.3.3 Sprint Planning

Sprint Planning involved selecting backlog items for implementation during a specific sprint and defining the sprint goal. During this phase, tasks were broken down into manageable development activities and assigned priorities based on project requirements. The purpose of sprint planning was to ensure that each development cycle focuses on delivering a clearly defined subset of system functionality while maintaining alignment with the overall project objectives.

### 3.3.4 Sprint Backlog and Development Sprints

The Sprint Backlog contained the selected backlog items scheduled for implementation during a sprint. Development was be organized into several iterative sprints, each focusing on a major subsystem of the ChainTix platform.

### 3.3.5 Daily Scrum

Daily Scrum meetings provided opportunities to monitor progress, identify development challenges, and coordinate ongoing activities. Although the project is developed by a single researcher, the principles of daily Scrum were applied through regular progress reviews and task tracking. These reviews helped ensure that development activities remained aligned with sprint goals and facilitate the early identification of implementation issues.

### 3.3.6 Sprint Review

At the conclusion of each sprint, the completed functionality was reviewed against the sprint objectives. The review process evaluates whether the implemented features satisfy the requirements specified during sprint planning. The outcomes of the sprint review informed subsequent development decisions and determined whether additional refinements are required before functionality is considered complete.

### 3.3.7 Sprint Retrospective

The Sprint Retrospective focused on evaluating the development process itself. During this stage, lessons learned, implementation challenges, and opportunities for improvement were identified and documented. Findings from the retrospective were used to refine development practices and improve the efficiency and quality of future sprints.

## 3.4 Analysis Diagrams

This section outlines the analysis models that were developed to capture the functional and structural requirements of the proposed ChainTix platform. These diagrams provided a blueprint for understanding user interactions, system behavior, and relationships among system components before implementation begins. The diagrams supported the Object-Oriented Analysis and Design (OOAD) paradigm adopted by the study and facilitated the translation of user requirements into technical specifications. The detailed diagrams are presented in Chapter Four.

### 3.4.1 Use Case Diagram

The Use Case Diagram was developed to illustrate the interactions between external actors and the ChainTix platform. It identified the major stakeholders of the system and the services

available to each stakeholder. The primary actors include the Event Organizer, Event Attendee, Gate Scanner Operator, and System Administrator. Through this diagram, the study modelled key functionalities such as event creation, ticket purchasing, ticket resale, ticket validation, attendance monitoring, and user management. By visualizing actor-system interactions, the diagram helped establish system boundaries and ensures that all functional requirements are adequately captured before implementation.

### 3.4.2 Class Diagram

The Class Diagram was used to represent the static structure of the proposed system by identifying the major classes, their attributes, methods, and relationships. Core classes within the architecture include User, Event, Ticket, Payment Transaction, Validation Token, and Smart Contract. The diagram illustrated associations, inheritance relationships, and dependencies among these classes, providing a clear representation of how data and functionality are distributed throughout the system. This model supported modular software design and serves as a foundation for the implementation of the system's object-oriented architecture.

### 3.4.3 Sequence Diagram

The Sequence Diagram was developed to model the chronological interactions that occur between system components during specific operations. Particular focus was placed on ticket purchasing and ticket validation workflows. The purchase sequence illustrated how requests flow between the web application, backend API, M-Pesa payment gateway, and blockchain network. The validation sequence illustrated the offline interaction between the attendee's browser-rendered credential, the gate operator's scanning interface, and the locally cached ticket data held in IndexedDB, demonstrating that no network round-trip occurs during gate validation. By representing the order of message exchanges and system responses, the Sequence Diagram will helped verify that all interactions occur in the correct sequence and that dependencies between components are properly managed.

### 3.4.4 Activity Diagram

The Activity Diagram was used to represent the workflow of key business processes within the ChainTix platform. Processes such as ticket purchasing, ticket resale, ticket validation, and event admission were modeled using activities, decision nodes, and process flows. The diagram highlighted alternative execution paths and conditions that may affect the outcome of a process.

This representation provided a high-level view of operational workflows and assisted in identifying potential bottlenecks, exceptions, and opportunities for process optimization.

### 3.4.5 Package Diagram

The Package Diagram was developed to organize the system into logical modules and illustrate dependencies between major software components. The architecture was grouped into packages including the Web Application Package, the Progressive Web Application Package containing the service worker, local storage service, and validation engine, the Backend Services Package, the Blockchain Package, the Payment Package, and the Validation Package. By grouping related classes and functionalities into cohesive units, the Package Diagram promoted modularity, improves maintainability, and provided a high-level overview of the overall system structure. The diagram also assisted in organizing the codebase and managing dependencies throughout implementation.

## 3.5 Design Diagrams

This section introduces the design models that guided the implementation of the proposed ChainTix architecture. While the analysis diagrams focus on requirements and system behavior, the design diagrams describe how the system was structured and deployed. These models provided a technical representation of the architecture and supported the realization of the functional and non-functional requirements identified during analysis. Detailed versions of the diagrams are presented in Chapter Four.

### 3.5.1 System Architecture Diagram

The System Architecture Diagram presented the high-level structure of the ChainTix platform and illustrate how its major components interact. The architecture consisted of a progressive web application serving three interfaces, namely the attendee ticket interface, the gate operator scanning interface, and the organiser dashboard; a FastAPI backend; a PostgreSQL database; smart contracts deployed on the Base blockchain; the M-Pesa Daraja payment gateway; and a client-side offline layer comprising service worker caching and IndexedDB storage on gate operator devices. The diagram showed the flow of information between these components, highlighting how payment processing, ticket issuance, ownership verification, and offline validation activities are coordinated within the platform. This architectural view provided a comprehensive understanding of the system's deployment environment and operational workflow.

### 3.5.2 Database Design

The Database Design model defined the structure of the relational database used by the system. It identified the entities, attributes, primary keys, foreign keys, and relationships necessary to support ticketing operations. Core entities include Users, Events, Tickets, Payments, Attendance Records, and Validation Tokens. The database design ensured efficient storage, retrieval, and management of application data while maintaining data integrity and consistency. Additionally, it provided the foundation for implementing business rules and supporting reporting functionality within the platform.

### 3.5.3 Wireframes

Wireframes will be developed to provide a visual representation of the user interfaces that comprise the ChainTix platform. These wireframes will depict the layout, navigation flow, and functionality of major system screens before full implementation begins. The designs will include interfaces for event browsing, ticket purchasing, ticket wallet management, ticket validation, and administrative operations. By providing a low-fidelity representation of the user experience, wireframes help validate interface requirements, improve usability, and guide frontend development. They also ensure consistency across the various applications that form part of the overall architecture.

## 3.6 Deliverables

The following deliverables constituted the output of the ChainTix project. Each deliverable corresponded to a self-contained system module or artefact, ensuring clear separation of concerns and facilitating independent testing and evaluation.

### 3.6.1 System Documentation

Documentation was developed to support the implementation, operation, and future maintenance of the proposed ChainTix platform. The documentation provided an overview of the system's structure, functionality, and usage, together with information necessary for deployment and administration. This deliverable served as a reference for users, developers, and other stakeholders involved in the system's operation and future enhancement.

### 3.6.2 Progressive Web Application

A progressive web application was developed to serve both event attendees and gate personnel through a single installable codebase. The attendee interface enabled users to browse events, purchase tickets using M-Pesa, view rotating time-based validation credentials, and manage ticket ownership including the optional transfer of ticket assets to self-custodial wallets. The

gate operator interface provided camera-based credential scanning with validation performed entirely on the client device, supported by service worker caching of the application shell and IndexedDB persistence of synchronised ticket data. The application was installable to the device home screen through a Web App Manifest and operated without network connectivity following a single pre-event synchronisation.

### 3.6.3 Smart Contracts

A set of blockchain smart contracts was developed and deployed on the Base blockchain network to manage ticket ownership and resale operations. These contracts automated the issuance of NFT-backed tickets, enforced resale restrictions, maintained immutable ownership records, and ensured transparency in ticket transfers. The smart contracts served as the foundation of the platform's trust model by reducing reliance on centralized intermediaries and providing verifiable ownership records for all issued tickets.

### 3.6.4 Administrative Dashboard

A web-based administrative dashboard will be developed to provide event organizers and system administrators with tools for managing platform operations. The dashboard supported activities such as event creation, ticket inventory management, attendance monitoring, sales tracking, and reporting. Through this interface, organizers were able to monitor ticket distribution, review transaction records, and obtain insights into event performance.

## 3.7 Tools and Techniques

This section describes the main tools and techniques that were used to develop the ChainTix platform. The selected technologies supported the implementation of the web interface, backend services, database layer, blockchain functionality, mobile-money payment integration, and offline validation mechanism.

### 3.7.1 React

React was used to develop all user-facing interfaces of the platform, comprising the attendee ticket interface, the gate operator scanning interface, and the organiser dashboard. React was appropriate for this project because it supports reusable interface components shared across the three interfaces, reducing duplication and ensuring visual consistency. It also integrates readily with the browser APIs required for offline operation, including the Service Worker registration lifecycle and asynchronous IndexedDB access, and its component-based state management is well suited to rendering credentials that update on a fixed thirty-second interval.

### 3.7.2 FastAPI

FastAPI was used to implement the backend API layer of the platform. It coordinated communication between the frontend applications, database, M-Pesa payment services, and blockchain components. FastAPI was suitable because it supports fast development of structured API endpoints and provides a lightweight framework for building scalable backend services (Mabotha et al., 2024).

### 3.7.3 PostgreSQL

PostgreSQL was used as the relational database management system for the platform. It stored application data such as users, events, tickets, payments, validation records, and transaction logs. Its relational structure is suitable for maintaining data consistency and supporting structured queries across the ticketing workflow.

### 3.7.4 Base Blockchain

The Base blockchain was used to support the decentralized ticket ownership layer of the system. It hosted the smart contracts responsible for NFT-backed ticket issuance and resale control. Base is suitable for the project because it provides an Ethereum-compatible environment while supporting lower-cost blockchain transactions compared to Ethereum mainnet (Base - Base Documentation, 2026).

### 3.7.5 Solidity

Solidity was used to develop the smart contracts that implement ticket ownership, minting, transfer, and resale restrictions. It was appropriate because it is the standard programming language for Ethereum-compatible smart contracts and is supported by the Base blockchain ecosystem.

### 3.7.6 M-Pesa Daraja API

The M-Pesa Daraja API was used to support mobile-money payment processing. It enabled users to purchase tickets using M-Pesa and allow the backend to verify payment transactions before issuing tickets. This tool was central to the project because mobile money is a key requirement for adapting blockchain ticketing to the Kenyan market.

### 3.7.7 Time-Based One-Time Password (TOTP)

TOTP was used as the offline ticket validation technique. It generated time-sensitive validation credentials that can be checked by authorized gate personnel without requiring continuous internet connectivity. This technique supported the project's goal of improving gate validation reliability in venues where network connectivity may be limited or unstable.

### 3.7.8 Web Crypto API

The Web Crypto API performed the cryptographic operations required for offline credential validation, specifically HMAC-SHA1 computation for time-based one-time password verification and ECDSA signature verification for confirming ticket authenticity against the backend's public key. As a native browser interface implemented in the underlying browser engine rather than in JavaScript, the Web Crypto API provides cryptographic performance and correctness guarantees comparable to native platform implementations, without introducing external library dependencies into the offline execution path.

# Chapter 4: System Analysis and Design

## 4.1 Introduction

This chapter presents the analysis and design of the ChainTix blockchain-based ticketing platform. It defines the system requirements and presents the Use Case, Class, Sequence, Activity, Package, System Architecture, Database, and Wireframe diagrams used to model the system. The design integrates conventional web technologies with blockchain-based ticket ownership, M-Pesa payments, smart contract enforcement, and offline ticket validation. These components are combined to provide a secure and reliable ticketing solution that addresses key challenges such as ticket fraud, scalping, and dependence on continuous internet connectivity.

## 4.2 System Requirements

The system functionalities are grouped into functional and non-functional requirements. These define how the system operates to meet user needs, as outlined below:

### 4.2.1 Functional Requirements

i. FR-0:1 User Registration and Authentication

The system allowed users to create accounts and securely authenticate before accessing protected functionality. Users were assigned appropriate roles, including Event Organizer, Event Attendee, Gate Scanner Operator, and System Administrator. Role-based access ensured that users can only access functionality relevant to their responsibilities.

ii. FR-02: Event Management Module

This module allowed event organisers to create and configure events, specifying the title, venue, date, ticket tiers, pricing in Kenyan Shillings, and total capacity. Organisers also defined the resale floor and ceiling prices and the royalty percentage for each ticket tier. These commercial parameters were written into the smart contract at deployment, ensuring they could not subsequently be altered by the platform, the organiser, or any other party.

iii. FR-03: Ticket Purchase and Payment Module

This module implemented the mobile-money purchase workflow. On ticket selection, the backend triggered a Lipa Na M-Pesa Online request to the buyer's handset through the Safaricom Daraja API. The asynchronous confirmation callback was validated against the originating checkout reference before the ticket was activated, and duplicate callbacks were rejected to prevent double processing. Failed or unconfirmed payments released the reserved ticket back to the available pool.

iv. FR-04: Blockchain Ticket Issuance Module

This module minted a non-fungible token representing each ticket to the buyer's custodial wallet on the Base blockchain following confirmed payment. Minting was queued and executed asynchronously so that payment confirmation was never delayed by blockchain transaction latency. The resulting token identifier, transaction hash and mint status were recorded against the ticket, establishing a publicly verifiable ownership record.

v. FR-05: Ticket Resale and Transfer

The system allowed eligible ticket owners to transfer or resell tickets subject to rules defined by the event organizer and enforced by the smart contract. The smart contract enforced applicable resale restrictions, including resale price limits where defined. This ensured that secondary-market transactions remain subject to predefined rules rather than being completely uncontrolled.

vi. FR-06: Dynamic Ticket Validation Credential

The system generate a time-sensitive validation credential for each ticket. The credential changed periodically to reduce the effectiveness of screenshots and copied ticket credentials. The architecture used TOTP as the offline validation mechanism, with browser-side cryptographic operations performed using the Web Crypto API.

### 4.2.2 Non-Functional Requirements

i. NFR-01: Security

The system protected user accounts, payment information, ticket ownership records, and validation credentials from unauthorized access. Authentication and authorization were implemented using role-based access controls. Cryptographic mechanisms were used to verify ticket authenticity, while blockchain records provided tamper-resistant ownership information.

ii. NFR-02: Availability

The platform remained usable despite temporary internet connectivity problems. In particular, the gate validation subsystem continued operating after the required data has been synchronized to the scanning device. This requirement was central to the architecture because the study specifically addresses environments where continuous connectivity at event venues cannot be guaranteed.

iii. NFR-03: Performance

The system shall process normal user operations within an acceptable response time. Ticket browsing, user authentication, and database operations should return responses without unnecessary delay, while payment and blockchain operations shall provide appropriate progress and status feedback.

iv. NFR-04: Usability

The system shall provide simple interfaces that can be used by attendees, event organizers, and gate operators with minimal technical expertise. The ticket purchasing and ticket validation workflows shall minimize unnecessary steps. The use of React across the interfaces supports reusable components and consistency between the attendee, scanner, and organizer interfaces.

v. NFR-05: Scalability

The architecture shall support increasing numbers of events, users, tickets, and validation operations without requiring a fundamental redesign of the system.

### 4.2.3 Data Requirements

Data requirements describe the information that the system collects, generates, processes, protects, and retains throughout the ticket lifecycle. These requirements cover all major stages of the platform, starting from user registration and ticket purchase, through payment processing and ticket validation, to post-event reconciliation. They are influenced by both the technical requirements of the system and the legal requirements governing personal and financial information in Kenya.

i. Data Inputs and Sources

The system receives data from five main sources. Attendees provide their mobile phone numbers and the tickets they wish to purchase. Organisers provide event information, ticket categories, prices, resale limits, royalty rates, and business verification details such as the registration number and KRA PIN.

Payment information is received from the Safaricom Daraja API. This includes the merchant request ID, checkout request ID, result code, M-Pesa receipt number, and transaction amount. The Base blockchain provides blockchain-related information such as contract addresses, token IDs, and transaction hashes. In addition to these external sources, the platform generates its own data, including unique identifiers, credential secrets, session tokens, and audit records.

ii. Data Formats and Validation Rules

The system applies specific formatting and validation rules to maintain consistency across the different types of data it processes. Mobile phone numbers are converted into the twelve-digit international format required by the Daraja API before they are stored. This helps ensure that payment callbacks can be correctly matched to the account that initiated the transaction.

Monetary values are stored as integers in Kenyan Shillings. This is appropriate for M-Pesa transactions, which do not use fractional currency units, and also avoids possible rounding errors when processing payments. Blockchain wallet addresses are stored as forty-two-character checksummed hexadecimal strings, while transaction hashes are stored as sixty-six-character hexadecimal strings in line with Ethereum address and transaction hash formats.

All timestamps are stored in Coordinated Universal Time (UTC). Local device time is only used when calculating credential validity windows and is not used for permanent record keeping. Resale prices are checked against the minimum and maximum limits configured for each ticket category. This validation is performed both by the application and by the smart contract to provide an additional level of protection.

iii. On-Chain and Off-Chain Data Separation

The system deliberately separates information stored on the blockchain from information stored in the private relational database. Only information necessary for blockchain-based ticket management is recorded on-chain. This includes the ticket token ID, owner wallet address, resale price ceiling, royalty percentage, and admission status.

Personal information is not stored on the public blockchain. This is particularly important because the Data Protection Act 2019 provides data subjects with rights relating to their personal information, including the right to erasure. Since records written to a blockchain are difficult to remove once they have been added to the ledger, storing personal information directly on-chain could create privacy and compliance challenges.

Instead, the system uses the custodial wallet address as a pseudonymous identifier. The relationship between the wallet address and the actual user is maintained in the PostgreSQL database, where the information can be managed and deleted when required.

### 4.3 System Analysis Diagrams

This section presents the analysis models developed for the ChainTix platform. The models describe the users of the system, system objects, process flows, and relationships between the different components.

### 4.3.1 Use-case Diagram

The use case diagram defines the system boundary and the services available to each actor. Four human actors are shown. The Event Attendee registers by phone number, browses events, buys tickets through M-Pesa, views the rotating credential, and resells tickets. The Gate Operator syncs ticket data and scans credentials at entry. The Event Organiser creates events, sets the resale limit and royalty rate, and views sales reports. The System Administrator manages accounts. The M-Pesa API and the NFT Blockchain appear as external systems on the boundary. Use cases are grouped into User and Access, Ticketing and Resale, and Administration and Reports.

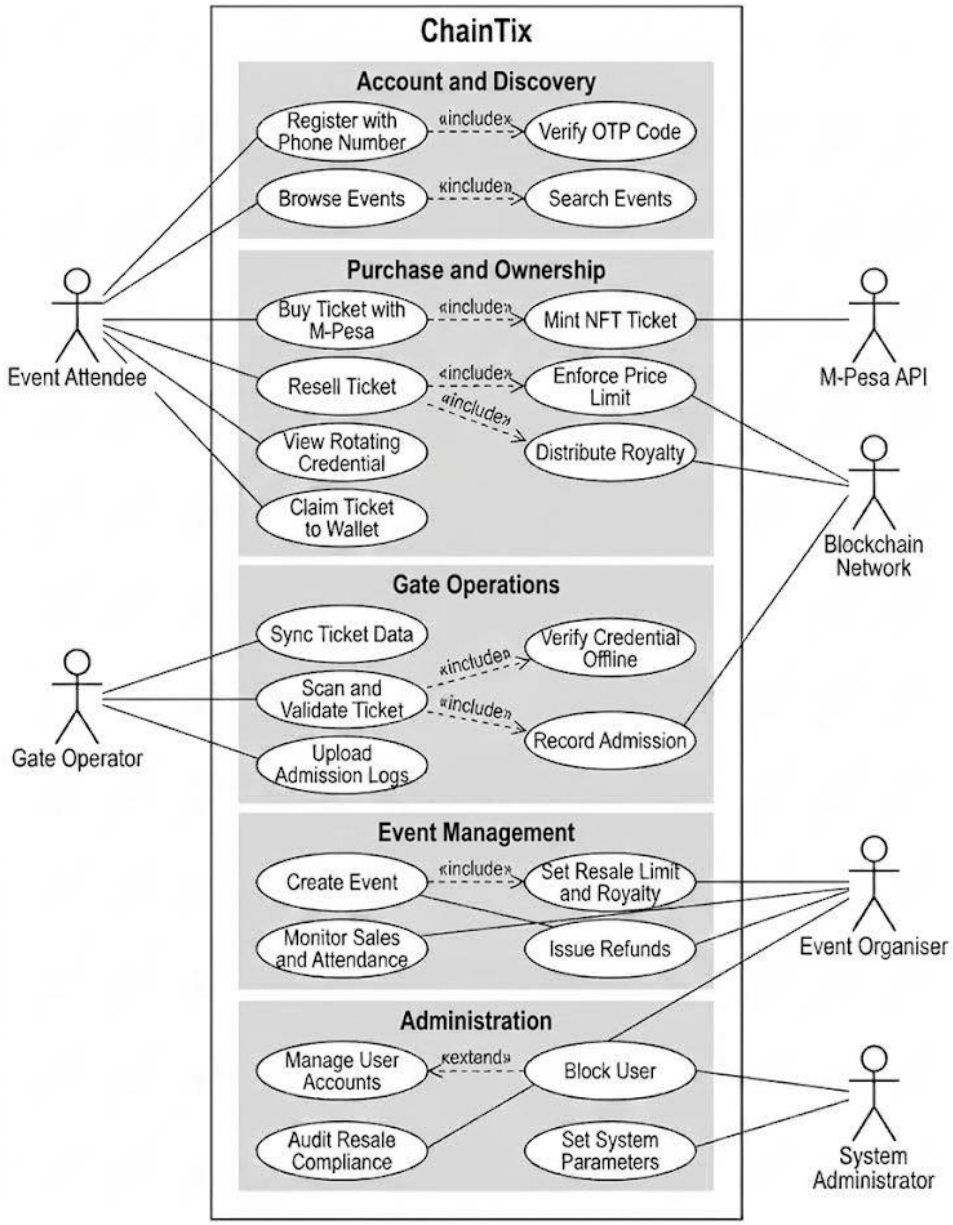



Figure 4.1 Use Case Diagram

The include relationships are the most important part of the diagram. Registration includes OTP verification, purchase includes NFT minting, and resale includes price limit enforcement. Because an included use case always executes, these are mandatory parts of each workflow rather than optional features the operator could disable. The extend relationships on gate scanning are conditional instead, reflecting that the validation path depends on connectivity at the venue. Placing M-Pesa and the blockchain outside the groupings shows that the platform mediates all access to both, so no attendee interacts with either directly.

### 4.3.2 Class Diagram

The class diagram shows the static structure of the system through its main classes, attributes, operations, and associations. The User class holds identity and wallet details. The Event class holds the price, capacity, resale multipliers, and royalty percentage. The Ticket class holds the credential secret, token identifier, transaction hash, and gate assignment. The Payment class records the transaction lifecycle, while the MpesaWebhookBridge class handles all Daraja interaction. The ValidationEngine and LocalStore classes sit on the operator device and perform offline verification. The SmartContract class governs minting, price cap enforcement, royalty distribution, and on-chain admission recording.

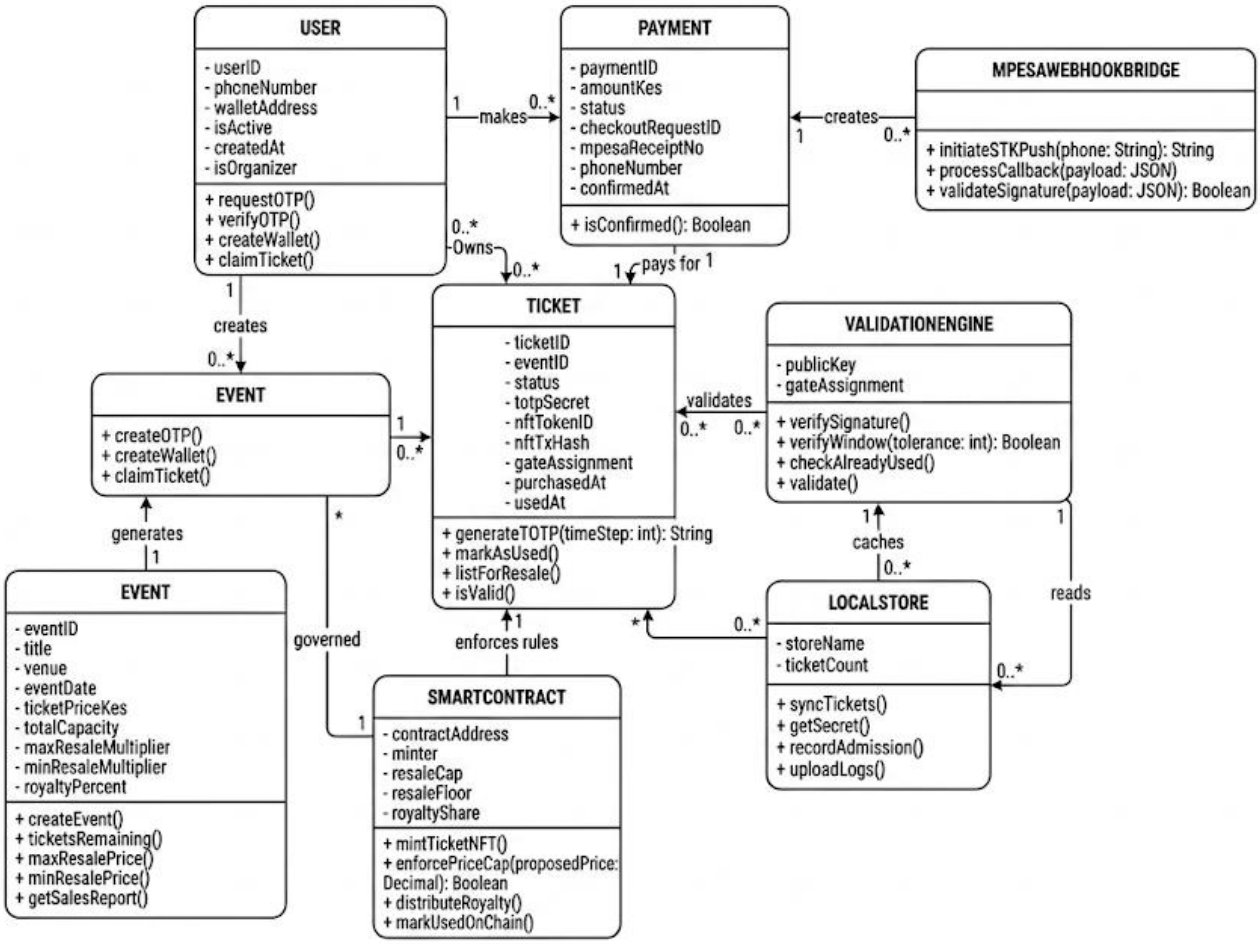



Figure 4.2 Class Diagram

The one-to-one association between Payment and Ticket is deliberate. It prevents a single confirmed transaction from issuing more than one ticket, which is the main integrity risk created by the asynchronous Daraja callback. The diagram also shows enforcement divided rather than concentrated: the SmartContract governs the Event rather than the reverse, so parameters set by an organiser become binding once deployed. Neither the ValidationEngine nor the LocalStore associates with Payment, Event, or MpesaWebhookBridge. That absence is what allows validation to run while the rest of the system is unreachable.

### 4.3.3 Sequence Diagram

The sequence diagram shows the ordered exchange of messages between nine participants across three phases. Phase one covers event creation, where the organiser submits event details and the smart contract is deployed. Phase two covers ticket purchase, from the M-Pesa push request through payment authorisation to NFT minting and credential delivery. Phase three covers gate validation, where the attendee interface computes a rotating credential and the

scanner verifies it against its local cache. Messages are numbered so the execution order can be traced without ambiguity.

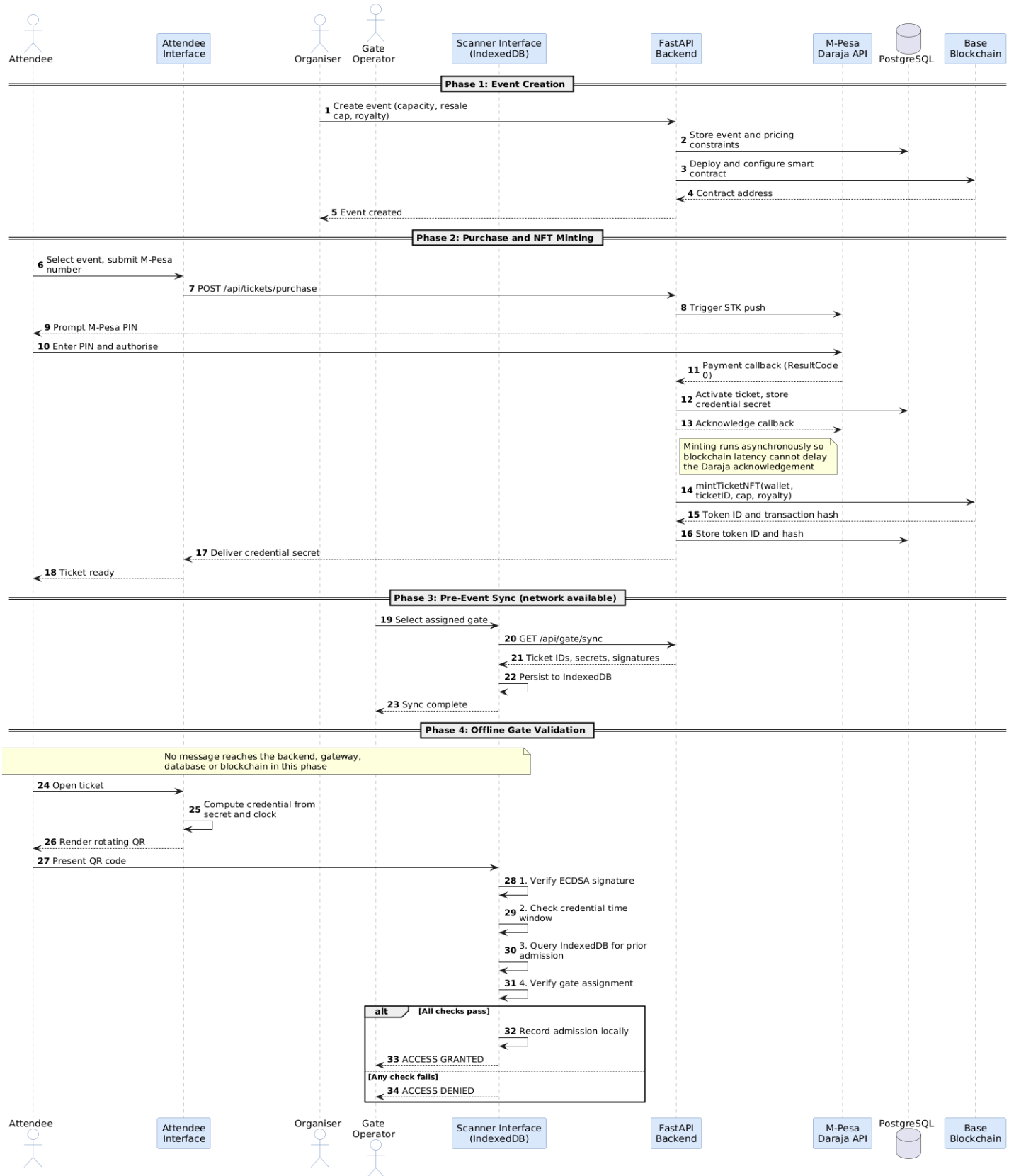



Figure 4.3 Sequence Diagram

The message ordering in phase two resolves a timing constraint. Daraja imposes a timeout on callback acknowledgement, while minting on Base can take longer than that timeout allows. Acknowledging the gateway independently of the mint prevents blockchain latency from recording a successful payment as failed. Phase three is the most consequential part of the diagram. All messages pass between the attendee interface, the scanner, and the local cache. The backend, payment module, database, and blockchain lifelines receive nothing, and the alternative fragment resolves entirely on the device. This is the visual proof that admission requires no network.

### 4.3.4 Activity Diagram

The activity diagram shows the operational workflow using four swimlanes that attribute each activity to the User, the System, the M-Pesa and Blockchain services, or the Database. The flow runs from event browsing and ticket selection through checkout, the M-Pesa push request, and the wait for the Daraja callback. A successful callback leads to NFT minting, credential seed generation, and ticket delivery, while a failed one terminates in a payment failed state. The resale path then validates a requested price against the smart contract cap, listing the ticket if it falls within the limit and rejecting it if it does not.

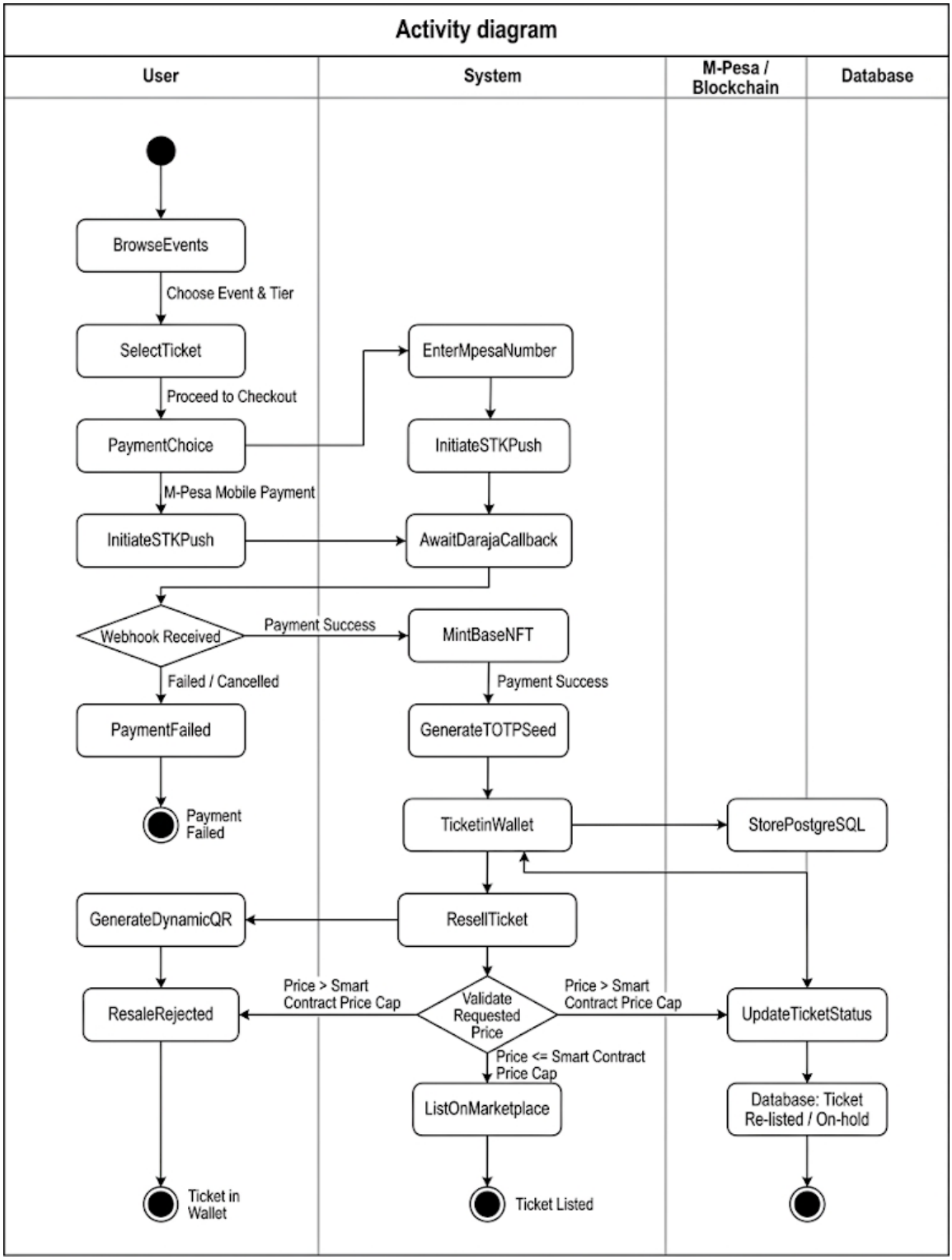



Figure 4.4 Activity Diagram

The two decision nodes each demonstrate a different principle. The webhook decision has branches that both terminate cleanly, so no path exists by which a ticket is issued before payment settles. The price validation decision is where anti-scalping enforcement occurs. Because the comparison runs against the smart contract cap rather than a database value, the

outcome cannot be altered by the platform operator or by anyone with database access. A rejected resale returns the ticket unchanged rather than voiding it, so attempting to list above the ceiling carries no penalty beyond refusal.

## 4.4 System Design Diagrams

### 4.4.1 System Architecture Diagram

The system architecture diagram shows the runtime structure of the platform across five layers. The presentation layer is a progressive web application serving three interfaces from a single codebase: the attendee interface, the organiser dashboard, and the gate scanning interface. The offline layer sits on the gate operator device and holds the service worker, the IndexedDB store, and the validation engine. The application layer holds the FastAPI routes, the service layer, and the asynchronous worker that handles minting. The data layer holds PostgreSQL and the task queue broker. The blockchain and external services layer holds the Daraja gateway, the deployed smart contract, and the Base network.

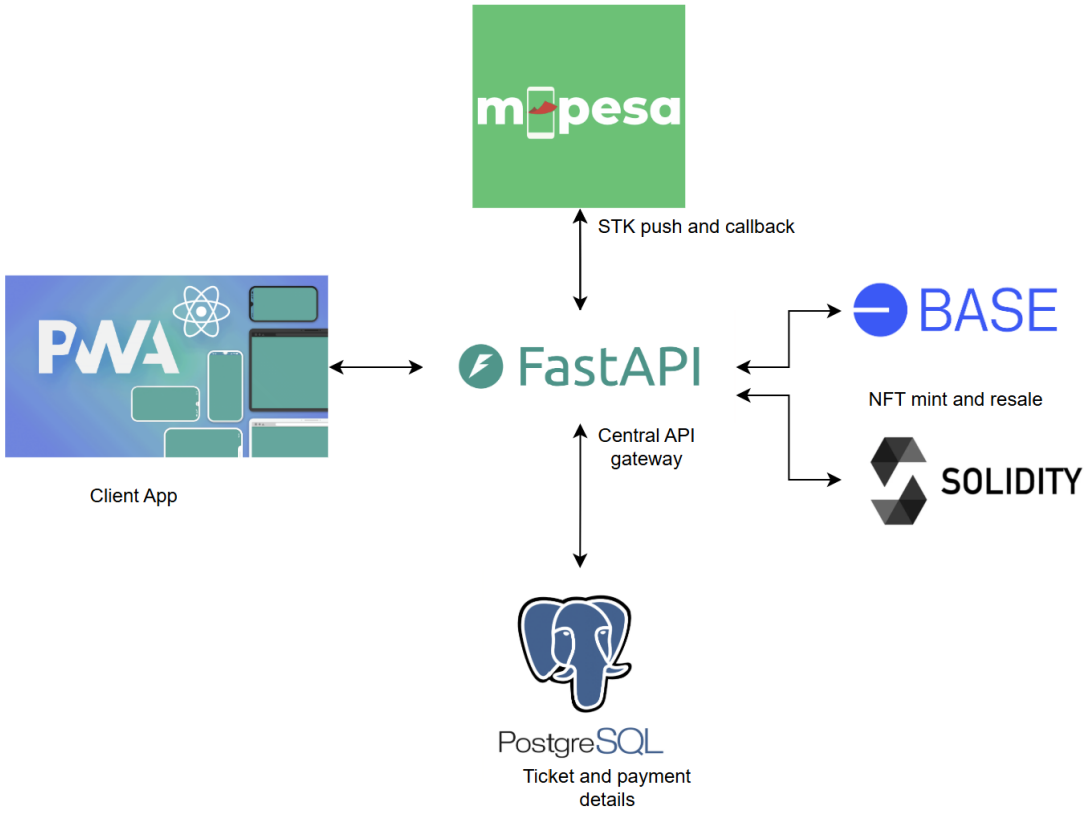



Figure 4.5 System Architecture Diagram

### 4.4.2 Database Design

The database schema defines the tables, keys, and relationships supporting the ticketing workflow. Fourteen tables are used. The accounts and organizers tables hold identity and business verification data. The events and event tiers tables hold the commercial configuration of each event. The tickets table holds each issued ticket together with its blockchain token identifier and transaction hash. The payments table records every transaction attempt from initiation to settlement or failure. The listings and transfers tables record secondary market activity, and the price caps table records each adjustment to the resale bounds. The barcodes table holds the versioned credentials issued across a ticket's lifetime. The gate scanners, notifications, and audit logs tables record device assignments, outbound messages, and system actions respectively.

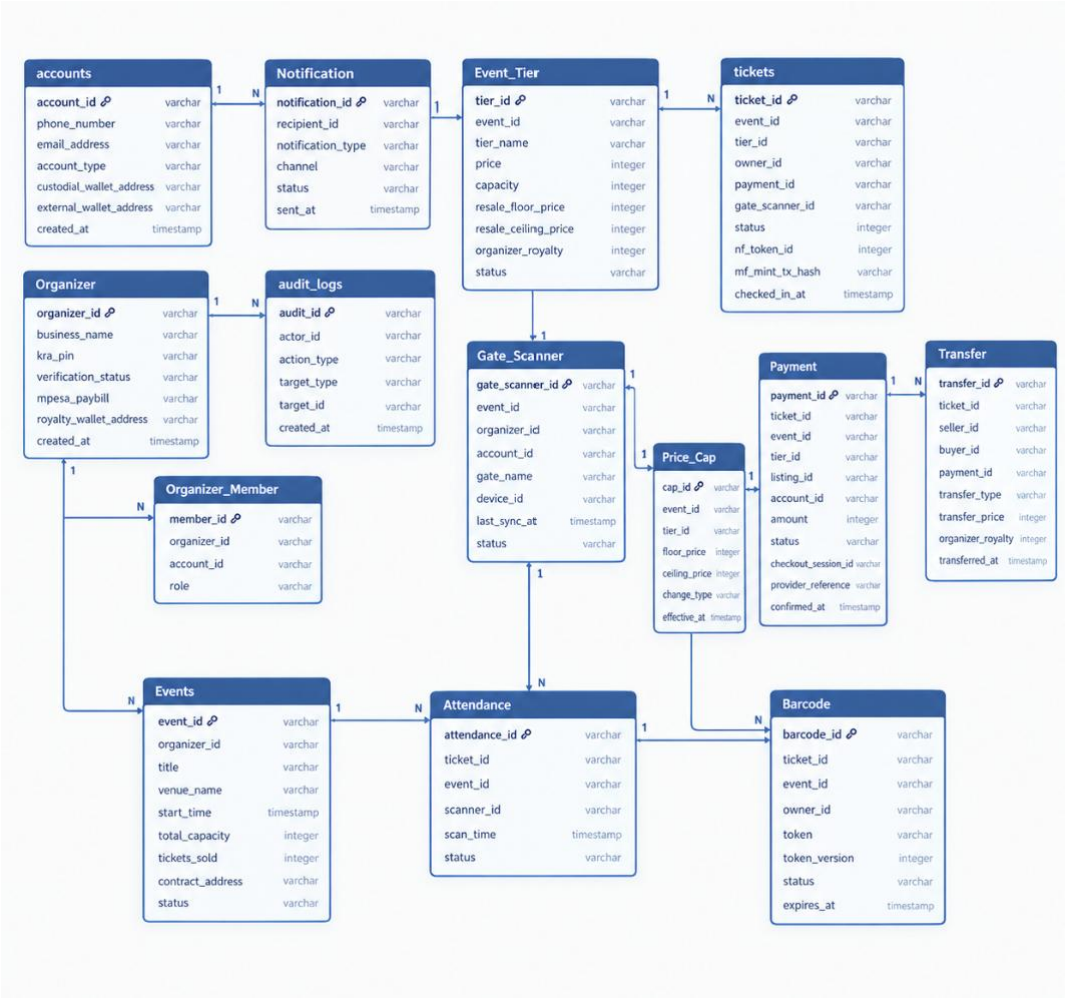



Figure 4.6 Database Schema

### 4.4.3 Wireframes

The event listing screen presents available events as cards showing the venue, date, and price in Kenyan Shillings. Pricing is displayed in local currency at every point in the interface, with no reference to cryptocurrency or token values anywhere in the attendee journey.

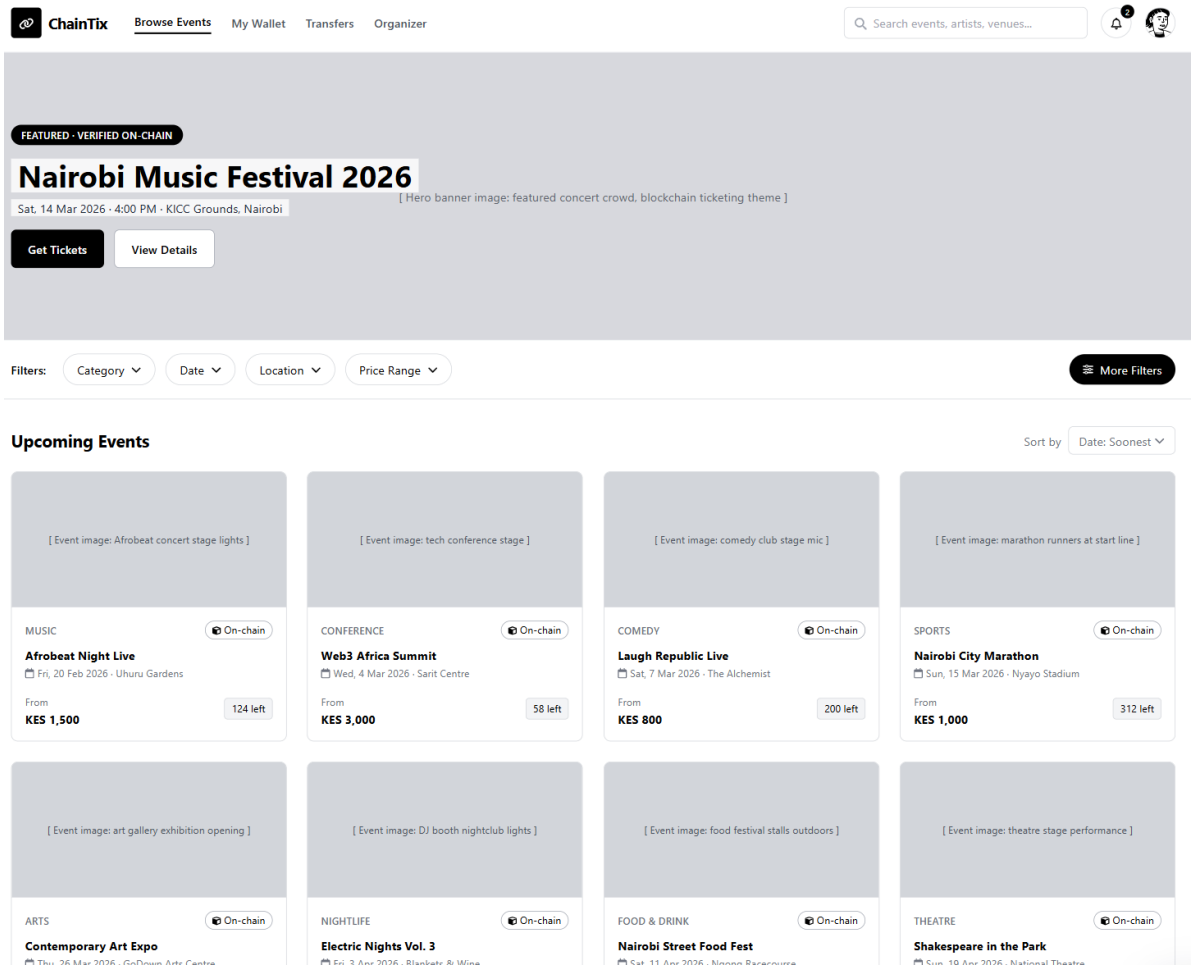
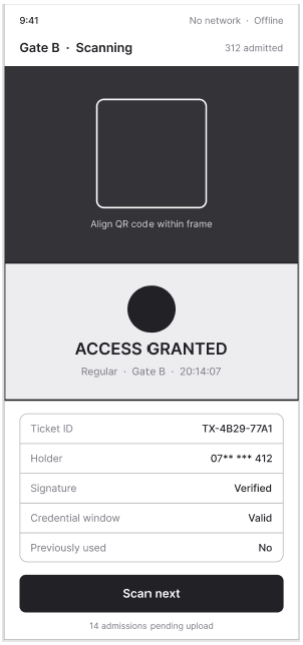
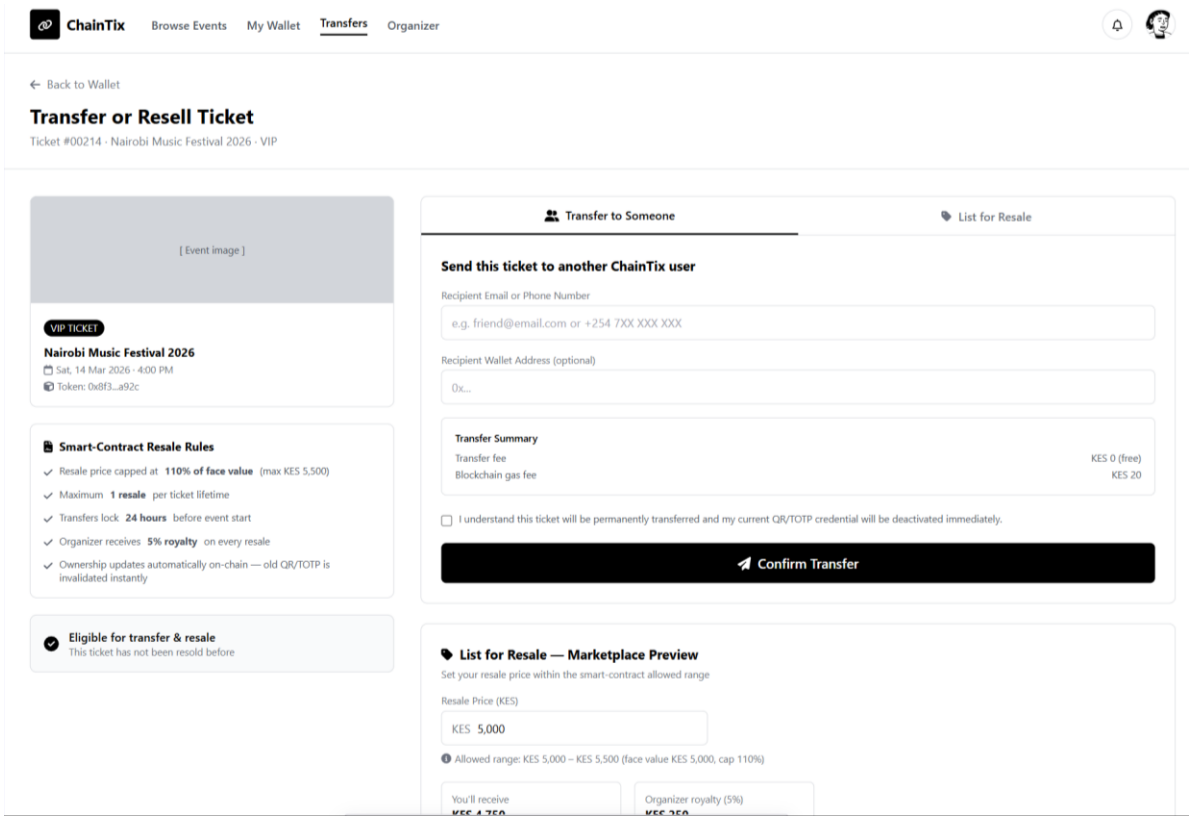



Figure 4.7 Event Listing Screen


9:41
No network · Offline
Gate B · Scanning
312 admitted
Align QR code within frame
ACCESS GRANTED
Regular · Gate B · 20:14:07
Ticket ID
TX-4B29-77A1
Holder
07** *** 412
Signature
Verified
Credential window
Valid
Previously used
No
Scan next
14 admissions pending upload


ChainTix
Browse Events
My Wallet
Transfers
Organizer
Back to Wallet
Transfer or Resell Ticket
Ticket #00214 · Nairobi Music Festival 2026 · VIP
[ Event image ]
VIP TICKET
Nairobi Music Festival 2026
Sat, 14 Mar 2026 · 4:00 PM
Token: 0x8f3...a92c
Smart-Contract Resale Rules
Resale price capped at 110% of face value (max KES 5,500)
Maximum 1 resale per ticket lifetime
Transfers lock 24 hours before event start
Organizer receives 5% royalty on every resale
Ownership updates automatically on-chain — old QR/TOTP is invalidated instantly
Eligible for transfer & resale
This ticket has not been resold before
Transfer to Someone
List for Resale
Send this ticket to another ChainTix user
Recipient Email or Phone Number
e.g. friend@email.com or +254 7XX XXX XXX
Recipient Wallet Address (optional)
0x...
Transfer Summary
Transfer fee
KES 0 (free)
Blockchain gas fee
KES 20
I understand this ticket will be permanently transferred and my current QR/TOTP credential will be deactivated immediately.
Confirm Transfer
List for Resale — Marketplace Preview
Set your resale price within the smart-contract allowed range
Resale Price (KES)
KES 5,000
Allowed range: KES 5,000 – KES 5,500 (face value KES 5,000, cap 110%)
You'll receive
Organizer royalty (5%)

## References


Alami, A., & Krancher, O. (2022). How Scrum adds value to achieving software quality? *Empirical Software Engineering*, *27*(7). https://doi.org/10.1007/S10664-022-10208-4

Audience View. (2026). *Fighting Fraud in Live Events*. https://audienceview.com/thought-leadership/fraud-in-the-live-event-industry/

Avalanche. (2025). *The Future of Ticketing | Avax.network*. https://www.avax.network/case-studies/the-future-of-ticketing

*Base - Base Documentation*. (2026). https://docs.base.org/get-started/base

Corsi, P., Lagorio, G., & Ribaudo, M. (2019). Ticketh, a ticketing system built on ethereum. *Proceedings of the ACM Symposium on Applied Computing*, *Part F147772*, 409–416. https://doi.org/10.1145/3297280.3297323

Dhanushkodi, R. M. (2025). A Blockchain-Based Generic and Reusable Ticket Booking Framework: Scalable Implementation Using AWS Cloud Infrastructure. *SSRN Electronic Journal*. https://doi.org/10.2139/SSRN.5283553

Gómez, B., Sánchez, K., & Salas, C. (2024). SafeTicket: Tickets Sale Safely Supported by Blockchain and NFTs. *Smart Innovation, Systems and Technologies*, *379*, 423–431. https://doi.org/10.1007/978-981-99-8612-5_34

Gülcüoğlu, E., Ustun, A. B., & Seyhan, N. (2021). Comparison of Flutter and React Native Platforms. *Journal of Internet Applications and Management*, *12*(2), 129–143. https://doi.org/10.34231/IUYD.888243

Mabotha, E., Mabunda, N. E., & Ali, A. (2024). Performance Evaluation of a Dynamic RESTful API Using FastAPI, Docker and Nginx. *IEEE Global Energy Conference 2024, GEC 2024*, 174–181. https://doi.org/10.1109/GEC61857.2024.10881712

Miguel Faustino Oliveira, J., Manuel da Costa Alves Marques, J., & Romano Supervisor, P. (2021). *Dynamic QR codes for Ticketing Systems Information Systems and Computer Engineering Examination Committee*.

OPEN Ticketing Ecosystem. (2025). *Ticket Explorer Overview.* https://docs.onopen.xyz/docs/ticket-explorer-overview

Safaricom PLC. (2024, December 5). *5/12/2024 - Safaricom's M-PESA Hits 34 Million Customers in Kenya*. https://www.safaricom.co.ke/media-center-landing/press-releases/safaricoms-m-pesa-hits-34-million-customers-in-kenya

Scrum.org. (2026). *What is Scrum? | Scrum.org*. https://www.scrum.org/resources/what-scrum-module

SNS Insider. (2025). *Online Event Ticketing Market Size, Share & Growth 2025-2032*. https://www.snsinsider.com/reports/online-event-ticketing-market-4359

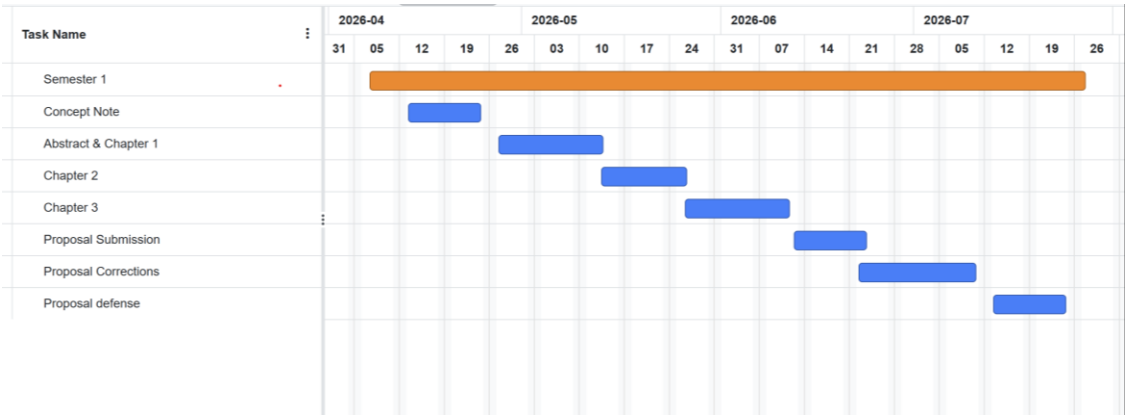
Soft Journ. (2026). *Preventing Ticketing Fraud in 2026: Technology and Strategies*. https://softjourn.com/insights/prevent-ticketing-fraud

SUTHIR, S., NIVETHITHA, V., & THANGAVEL, M. (2024). Enhancing Digital Identity and Access Control in Event Management Systems Using Sui Blockchain. *IEEE Access*, 14. https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=10872968

Tiko Hub. (2025). *Why Burna Boy's Concert Had Empty Seats: The Impact of Scalpers and Fake Tickets on Live Events*. https://www.tikohub.co.ke/post.php?id=23&slug=why-burna-boys-concert-had-empty-seats-the-impact-of-scalpers-and-fake-tickets-on-live-events

# Appendix:

## Appendix 1: Gantt Chart




## Appendix 2: Similarity Report

turnitin Page 2 of 40 - Integrity Overview Submission ID trn:oid:::2945:392991137

### 20% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

**Filtered from the Report**

- Bibliography
- Quoted Text

**Match Groups**

- **99** Not Cited or Quoted 18%
  Matches with neither in-text citation nor quotation marks
- **8** Missing Quotations 1%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

**Top Sources**

- 7% Internet sources
- 3% Publications
- 19% Submitted works (Student Papers)